

SmartStudy Cloud Agent

INFO-H505 – Cloud Computing – May 2026

Amine Bâaj Komildzhon Samadov Kevin Close Soulaïmane Rahmoune

Repository: <https://github.com/KevinClose/SmartStudy-Cloud-Agent>

Live demo: <https://smartstudy-app-930005856895.europe-west1.run.app>

1 Project Overview

SmartStudy is a cloud-native AI tutor that helps university students prepare for exams. The student uploads PDF course materials (slides, syllabuses, notes), and an AI agent answers questions about the content. The tutor only uses the uploaded documents, cites the source file and page for every claim, and ends each answer with a short pedagogical question to check what the student understood. The project is the implementation version of the Qwiklabs lab *Creating a RAG Chat Assistant with MongoDB Atlas Vector Search, Google Cloud and Langchain*: the original lab was a manual prototype written in JavaScript, ours is a fully automated Python event-driven system deployed on Google Cloud.

2 System Architecture

The system has two flows that run independently and share the same vector store and embedding model (Figure 1).

Ingestion (orange). The student uploads a PDF through the Streamlit sidebar; it lands in a dedicated Cloud Storage bucket. A storage event is automatically routed by Eventarc to a Cloud Function (second generation, Python runtime). The function downloads the file, extracts the text page by page, splits each page into overlapping chunks of about one thousand characters, embeds every chunk with the Vertex AI text embeddings model, and stores them in MongoDB Atlas together with their metadata (source file name and page number). Nothing has to be triggered manually.

Chat (blue). The student asks a question on the Streamlit interface deployed on Cloud Run. The question is embedded with the same model and used to query the Atlas Vector Search index using cosine similarity. The five most relevant chunks are reformatted with a citation made of the source file name and the page number, injected into the system prompt, and sent to Gemini 2.5 Flash. The answer is shown to the student and the full conversation is saved in a separate collection that keeps the history of every session.

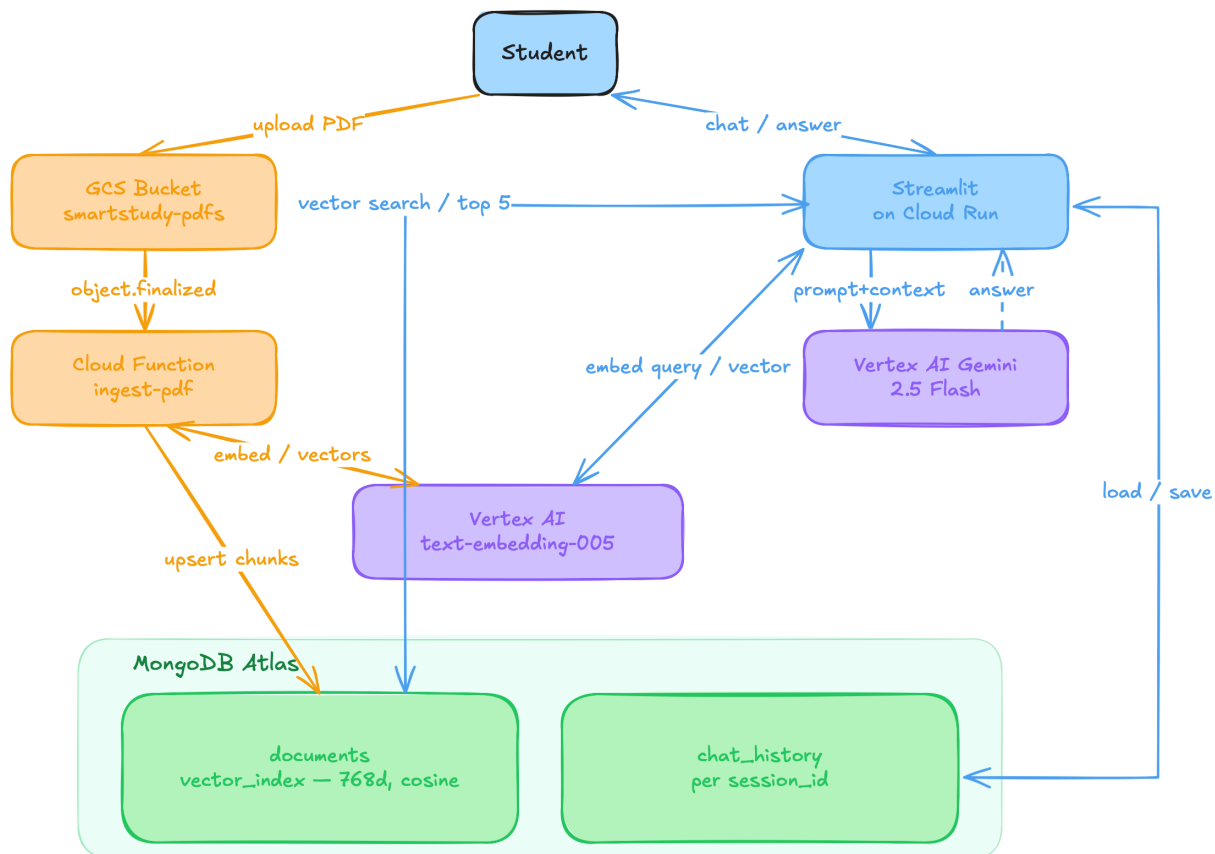


Figure 1: SmartStudy architecture: PDF ingestion (orange) and RAG chat (blue).

3 Implementation Choices and Changes from the Lab

Full Python rewrite. The whole codebase was rewritten in Python, both for the ingestion function and for the chat interface. The Python ecosystem makes the integration with Vertex AI, MongoDB Atlas and LangChain much more uniform than the original mix of JavaScript on the server side and a separate frontend framework.

From manual to automatic ingestion. This is the biggest change compared to the lab. In the original lab the student had to run a local script every time a new document had to be imported. In our version the Cloud Function is triggered automatically by the upload event emitted by Cloud Storage, so the student never has to launch a command. Setting up this automation took quite a few iterations because several internal Google service identities had to be authorised to talk to each other (the event routing service had to gain read access on the bucket, the storage service had to be allowed to publish notifications, and the default compute identity had to be allowed to invoke Cloud Run and to act on behalf of the project service account).

Cosine similarity instead of euclidean distance. The lab uses euclidean distance for the vector index. We switched to cosine similarity, which is more stable for modern text embeddings because the meaning is encoded in the direction of the vector rather than in its magnitude. The retrieval is less affected by the length of a chunk.

Streamlit interface on Cloud Run. The lab application only ran on the developer’s laptop. We built a real web interface with Streamlit, packaged it in a Docker image, and deployed it on Cloud Run. The service scales automatically, drops to zero instances when nobody is using it, and is reachable through a public HTTPS link. We also limited the maximum number of running instances so the project credit stays under control even if the URL is shared widely.

Conversation memory. The conversation history is the advanced feature we chose to integrate. Each session in the browser receives a unique identifier and every message exchanged is saved in a dedicated

MongoDB collection. The tutor reloads the previous messages on every new question, which makes natural follow-up exchanges possible (the student can ask the tutor to elaborate or to give another example without repeating the context).

Tutor persona and source citations. The system prompt enforces four important rules. The tutor must answer only from the retrieved content and must explicitly say so when the answer is not in the documents. Every factual statement must be followed by a citation that contains the source file name and the page number printed on the PDF. The answer must end with a short pedagogical follow-up question. Finally, off-topic questions must be redirected back to the course materials. The prompt is written in English because modern large language models follow English instructions more reliably; an explicit directive tells the model to answer in the language of the student question, so French questions still receive French answers with the same persona.

Idempotent re-ingestion. If the same document is uploaded a second time (for instance after a correction), the function first removes the existing chunks linked to that file name before inserting the new ones. Without this step, duplicates would slowly accumulate in the database and the retrieval would start mixing outdated content with the latest version.

Modular RAG chain. The retrieval and generation pipeline is built with the declarative chaining style provided by LangChain. Each step (retrieval, formatting, prompt building, model call, output parsing) is composed with a pipe operator, which makes the flow easy to read and to extend. The chain also preserves the intermediate values, which is what allows the interface to display the source chunks under every answer.

4 Team Workflow and Group Challenges

Working as a group of four was the part of the project that surprised us the most. Most cloud services we used require an active account or a billing setup that cannot easily be shared, and several deployments are tied to a specific user identity. As a result, it was not always practical for two of us to develop in parallel on the same feature.

In practice, each team member started by working on a separate feature on their own machine (the ingestion function, the Streamlit interface, the prompt design, the MongoDB indexes, the deployment scripts, the architecture diagram, the report). When a piece was ready, we met in person and we gathered around the laptop of the team member who was logged into all the required accounts at the same time. We then assembled the pieces together, made sure they worked end to end on that single environment, and pushed the result on GitHub. This was less efficient than parallel development, but it was the most reliable way to avoid permission errors, account conflicts and broken deployments. It also forced us to discuss every change together, which made the final integration much smoother than expected.

5 Limitations

Single-tenant design. All users share the same document library and the same vector index. Only the chat history is isolated per session through a session identifier. A real product would need a proper authentication layer and a per-user filter applied to every retrieval and storage operation.

Cost sensitivity. The generation model is billed per token. A malicious user who finds the public URL could in theory send many questions in a loop and consume the project credit. We limited the maximum number of Cloud Run instances and set up a billing alert, but the only fully safe option would be to add authentication on the application.

Free-tier limits. The free MongoDB Atlas cluster is limited to a small amount of storage and a small number of concurrent connections. It is enough for the demo and for a small course library, but a serious deployment would need a paid tier.

6 Conclusion

SmartStudy meets the four required features (bucket ingestion, automated cloud pipeline, RAG orchestration, tutor persona with citations) and adds two advanced features (a web interface deployed on Cloud Run and a persistent conversation memory). Compared to the original lab, the project moved from a manual JavaScript prototype to an automated Python cloud system with a public web interface. The full source code and the live demo are available through the links at the top of this report.